

NYCU-ECE DCS-2026

Lab05

Design: Password Lock

Data Preparation

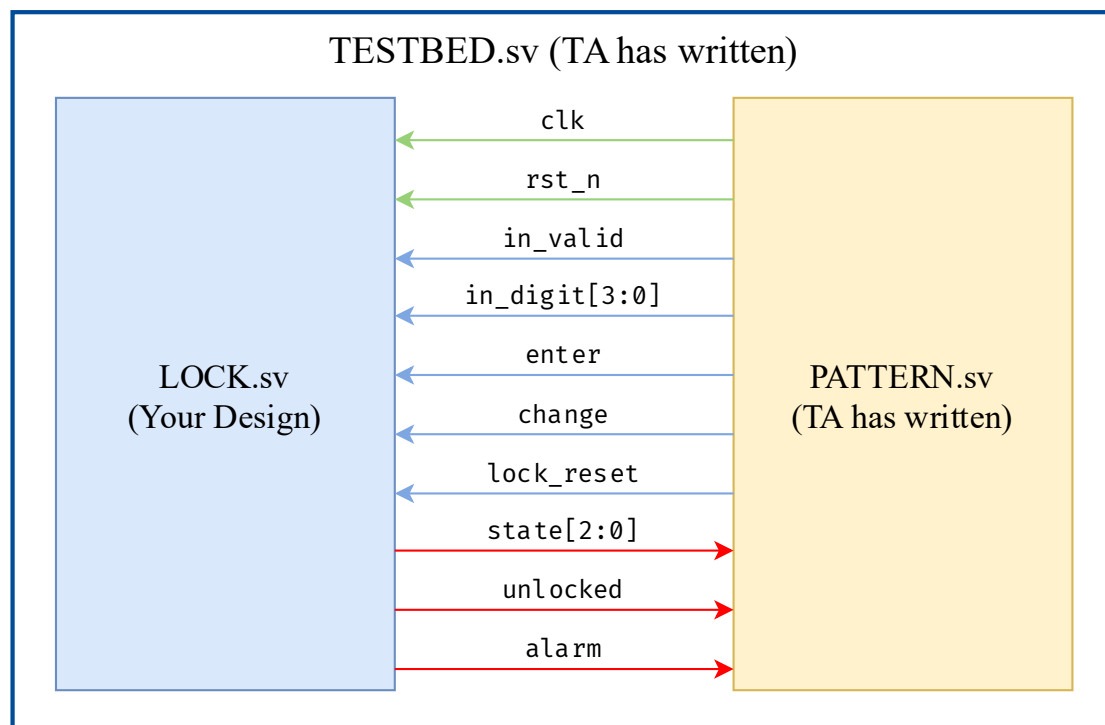
1. Extract the files needed from TA's directory:

```
$ tar -xvf ~es26dcsTA01/Lab05.tar
```

2. The extracted files contain:

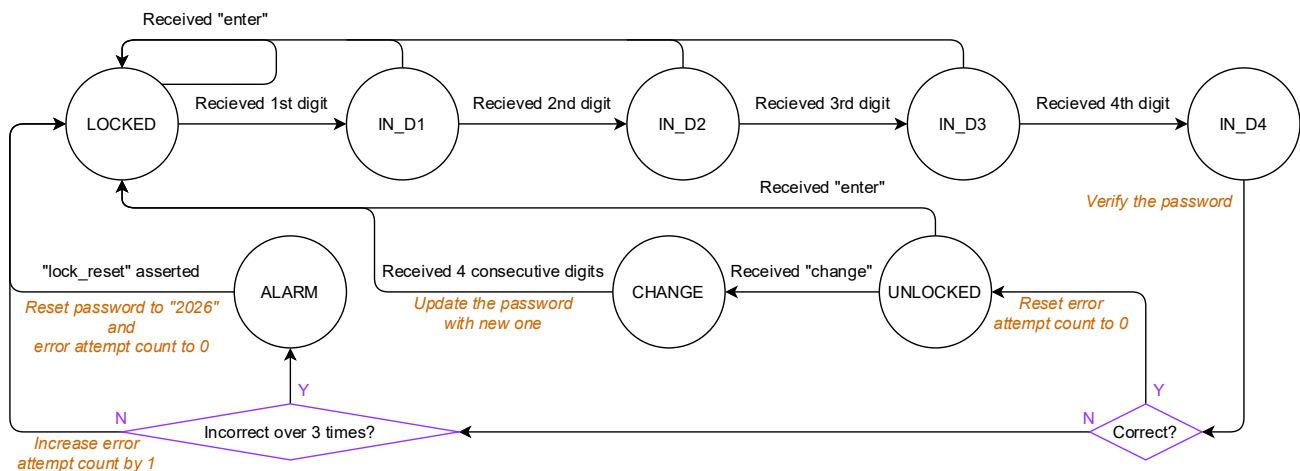
```
Lab05/  
├── 00_TESTBED/  
├── 01_RTL/  
├── 02_SYN/  
├── 03_GATE/  
└── 09_UPLOAD/
```

Block Diagram



Design Description

In this lab, you need to implement a 4-digit password lock FSM with password-change and alarm functionalities based on the provided state transition diagram.



After system reset, the lock defaults to the password “2026” and starts in the LOCKED state. For each state:

- **LOCKED/IN_D1/IN_D2/IN_D3:**

The system transitions sequentially through these states upon receiving digits. Pressing “enter” before four digits are entered returns the system to LOCKED without counting as an error attempt.

- **IN_D4:**

On the next clock cycle, change state based on the four input digits.

If the digits match the current password, transition to UNLOCKED. Otherwise, check the error attempt counter.

If the number of error attempts has reached 3, enter ALARM mode and lock the system. If it is less than 3, increment the error attempt counter and return to LOCKED.

- **UNLOCKED:**

If “enter” is received, transition back to LOCKED and clear the error attempt counter to complete the operation.

If “change” is received, transition to the CHANGE state.

- **CHANGE:**

Once valid data is available, the new 4-digit password is provided over 4 consecutive clock cycles. After receiving all four digits, update the stored password and return to LOCKED.

- **ALARM:**

The system is locked. Only when “lock_reset” is asserted should the FSM transition back to LOCKED, restore the default password 2026, and clear the error attempt counter.

The signals in_digit, enter, change, and lock_reset are considered valid only when in_valid is high. All valid data lasts only for exactly one cycle.

enter, change, and lock_reset are always not unknown. They would not be high if in_valid is low.

The four input digits for normal password entry do not guarantee to be in consecutive cycles. In the CHANGE state, however, once in_valid is asserted, the four digits of the new password must be provided in four consecutive cycles.

Inputs

Signal name	Bit width	Description
clk	1	Clock signal
rst_n	1	Asynchronous active-low reset signal
in_valid	1	Asserted when any one of the below 4 input signals is valid
in_digit	4	Input digit ranged from 0 to 9
enter	1	“Enter” key for user confirmation
change	1	Asserted to initiate password change mode
lock_reset	1	Asserted to reset the lock status

Outputs

Signal name	Bit width	Description
state	3	State for the password lock FSM 000: LOCKED 001: IN_D1 010: IN_D2 011: IN_D3 100: IN_D4 101: UNLOCKED 110: CHANGE 111: ALARM
unlocked	1	Raised when unlocked
alarm	1	Raise when triggering alarm state

Specifications

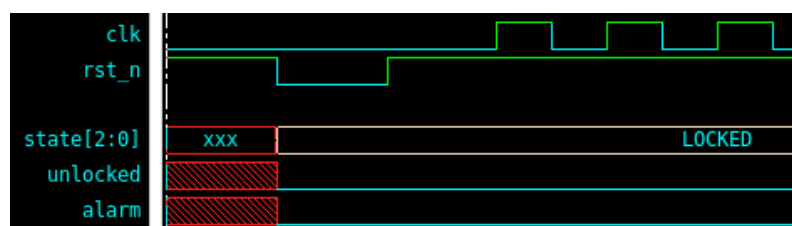
1. Top module name: **LOCK** (File name: **LOCK.sv**).
2. After asynchronous active-low reset, all output signals must be reset to zero.
3. unlocked is only raised in UNLOCKED state.
4. alarm is only raised in ALARM state.
5. State transition behavior must be **exactly identical** to the provided diagram.
6. The 02_SYN result **must not have any errors** and **must not contain any latches**.
7. The slack in the timing report must be non-negative, and the result must be “Met.”
8. Gate level simulation **must not have any timing violation**.
9. Clock period is 10.0 ns.
10. Input delay = $0.5 \times$ clock period; output delay = $0.5 \times$ clock period.
11. The design must implement the required functionality. **Do not design specifically for the test patterns**, such as determining it is the n -th pattern and explicitly setting the output. Such designs will be judged as fail during the demo.
12. Do not use `*error*`, `*latch*`, `*congratulation*` or `*fail*` as names for logic / wire / reg / submodule / parameter, otherwise the demo result will fail.
Note: `*` represents any symbol before or after the word. E.g., `error_test` is prohibited.
13. It is recommended to separate combinational blocks and sequential blocks.
14. **For loops are forbidden in this homework.**

Hints

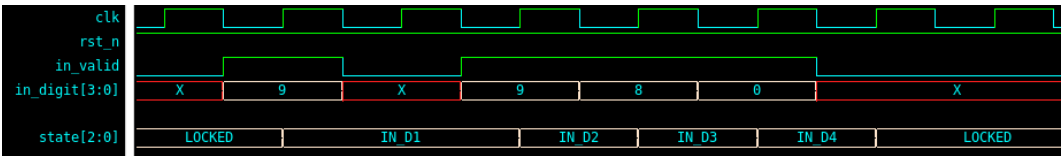
You may need **two register arrays** to store the input password and the correct password. Also, **two counters** may be useful to count the current digit position in CHANGE state and the number of error attempts.

Example waveform

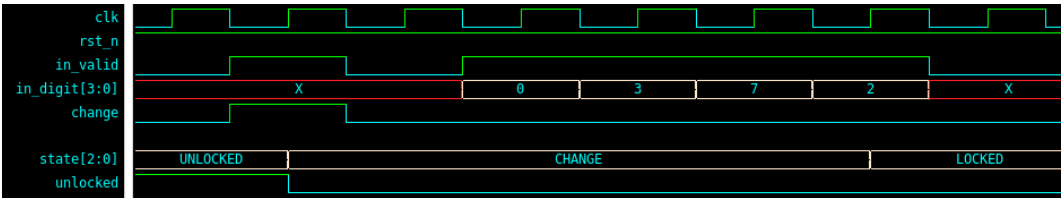
Reset all output after asynchronous and active-low reset signal.



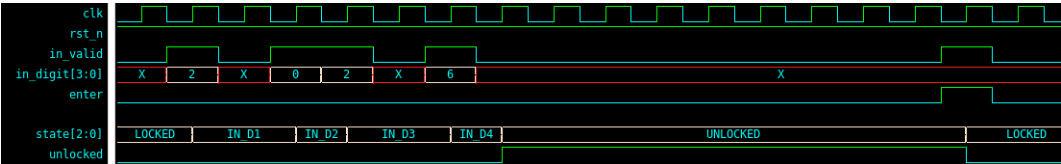
in_digit may be consecutive or intermittent.



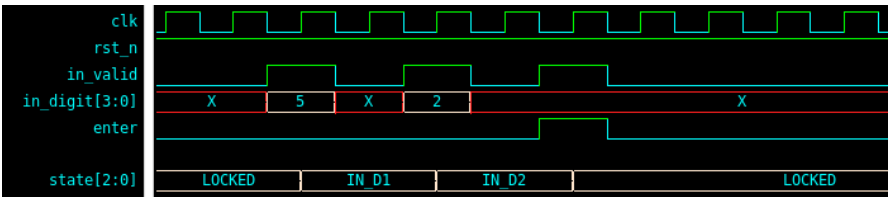
in_digit must be consecutive after change. (password change to “0372.”)



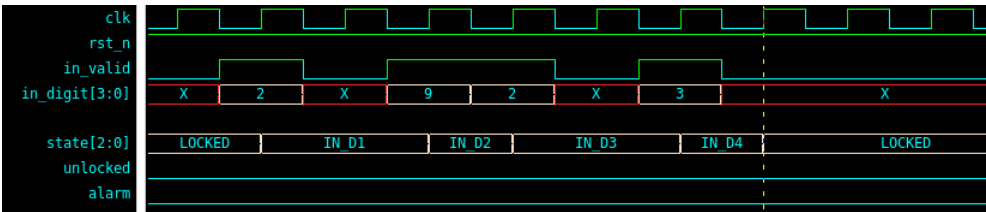
System unlocked with correct password “2026.”



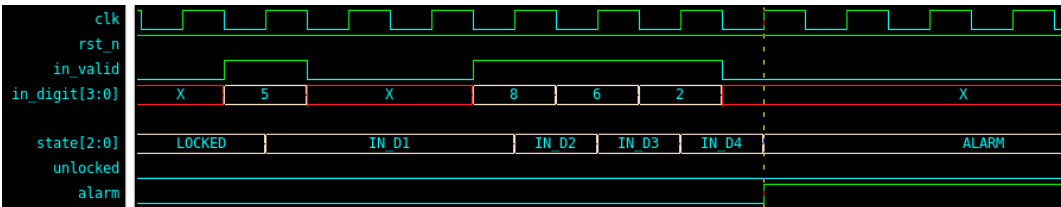
state returns to LOCKED if enter is detected early. (Do not increase error attempt.)



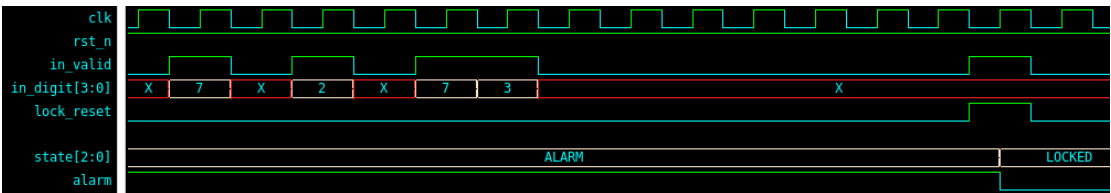
Incorrect password when attempts fail less than 3 times. System returns to LOCKED.



Incorrect password when 3 attempts fail continuously. Enter ALARM mode.



After entering ALARM mode, keep the state until lock_reset is triggered.



Upload file

- Use **./01_upload** under 09_UPLOAD/ to submit your design.
- Note that the script requires time to synchronize with the NTP server. Please avoid last-minute submissions.
- 1de deadline: TBD (tentatively 4/9 17:40); 2de deadline: 4/9 23:59.

Grading policy

- Pass the RTL, Synthesis, and GATE level simulation in 1st demo will get 100%.
- Pass the RTL, Synthesis, and GATE level simulation in 2nd demo will get 60%.

Note

Template folders and reference commands:

- 01_RTL/ (RTL simulation) → **./01_run_vcs_rtl**
- 02_SYN/ (synthesis) → **./01_run_dc**
- 03_GATE/ (gate-level simulation) → **./01_run_vcs_gate**
- 09_UPLOAD/ (design upload) → **./01_upload**